

UNIVERSIDAD DIEGO PORTALES
FACULTAD DE INGENIERÍA Y CIENCIAS
ESCUELA DE INFORMÁTICA Y TELECOMUNICACIONES



Sistemas Distribuidos
Informe

Profesor:
Nicolas Hidalgo

Estudiantes:
Ricardo Lepin, Pablo Muñoz

14 de septiembre de 2026

Índice

1. Introducción	1
2. Arquitectura del Sistema	1
3. Implementación de Servicios (Desarrollo Actual)	1
3.1. Generador de Tráfico	1
3.2. Sistema de Caché	1
3.3. Scraper Service	2
3.4. Metrics Service	2
4. Decisiones de Diseño y Tecnologías	2
5. Pruebas y Resultados Preliminares	2
6. Pruebas y Resultados Finales	4
6.1. Impacto de la Distribución de Tráfico y Tamaño de Caché (Uniforme)	4
6.2. Impacto de la Distribución de Tráfico y Tamaño de Caché (Zipf)	5
6.3. Comparación: Distribución Uniforme vs. Distribución de Zipf	5
6.4. Evaluación del Impacto del Time To Live	6
6.5. Validación Funcional y Resiliencia	7
6.6. Evidencia de las consultas Q1–Q5	7
6.7. Registro de Archivos de Experimentación	10
6.8. Métricas Complementarias de Rendimiento	11
6.9. Trazabilidad de Resultados Finales	11
7. Conclusiones	11

1. Introducción

Un sistema distribuido mal optimizado y planificado puede generar tiempos de espera prolongados y cuellos de botella. Sin embargo, al diseñar una arquitectura eficiente desde un principio, estos problemas son evitables.

Para optimizar los tiempos de respuesta y manejar eficientemente la concurrencia en la obtención de datos, este proyecto propone segmentar el sistema en múltiples servicios independientes. Estos módulos interactúan entre sí a través de la red mediante el uso de tecnologías como FastAPI, Redis y Docker, cumpliendo así con el objetivo de resolver consultas simuladas priorizando el uso de una caché en memoria antes de recurrir a procesos más costosos como el *web scraping*.

2. Arquitectura del Sistema

La arquitectura del sistema esta conformada por cuatro microservicios, que estan contenidos en un dockers para poder ejecutarse en distintos entornos sin problemas

El flujo del sistema comienza cuando el Generador de Tráfico genera una consulta hacia el Sistema de Caché. Este procesa la llave y verifica su existencia en Redis. Si los datos están disponibles (*Cache Hit*), Redis los retorna instantáneamente al usuario. En caso contrario (*Cache Miss*), la petición es delegada al Scraper Service, el cual extrae la información en tiempo real de la fuente, la devuelve a la caché para su almacenamiento con un Tiempo de Vida definido, y finalmente se entrega la respuesta. Todo este ciclo es monitoreado de manera asíncrona por Metrics Service para su posterior análisis.

3. Implementación de Servicios (Desarrollo Actual)

3.1. Generador de Tráfico

El servicio *Generador de Tráfico* tiene como objetivo principal simular el comportamiento concurrente de usuarios reales. Esto se logra enviando un flujo continuo de peticiones parametrizadas al sistema central para evaluar tanto su rendimiento general como el comportamiento específico de la memoria caché.

Para asegurar que la simulación represente un entorno real de uso, se implementó una distribución matemática de Zipf. Esta distribución garantiza que ciertas consultas se repitan con mayor frecuencia que otras, lo cual es un escenario ideal y necesario para poner a prueba la tasa de aciertos (*hit rate*) de nuestro sistema de caché.

Adicionalmente, el módulo fue diseñado con resiliencia frente a fallos de red. Se implementó un manejo de excepciones HTTP que permite al generador continuar su ciclo de peticiones de manera ininterrumpida, incluso si el servicio de caché de destino se encuentra momentáneamente inactivo o es inalcanzable, evitando así el colapso del módulo generador.

3.2. Sistema de Caché

El Sistema de Caché actúa como una memoria temporal que intercepta las peticiones para reducir la carga del *Scraper* y disminuir la latencia. Este módulo está construido con FastAPI debido a su alto rendimiento y está respaldado por una base de datos en memoria utilizando Redis.

En este sistema se diseñó un algoritmo alojado en el archivo `key_builder.py` para transformar los *payloads* JSON entrantes en llaves únicas. Este proceso de normalización no solo extrae el tipo de consulta, sino que también formatea los nombres reemplazando los espacios por guiones bajos. Además, incorpora lógicas específicas, como el ordenamiento alfabético de los equipos en consultas de historiales, lo que garantiza que peticiones equivalentes generen exactamente la misma llave, evitando así el almacenamiento de datos duplicados.

Al recibir una consulta, el sistema verificará su existencia en Redis. En caso de producirse un *cache hit*, retornará los datos inmediatamente. Por el contrario, si ocurriera un *cache miss*, se delegará la extracción de los datos y se guardará la respuesta en la base de datos con un Tiempo de Vida (TTL) preestablecido.

3.3. Scraper Service

El Scraper Service, implementado mediante FastAPI, es el componente encargado del trabajo de recolección de datos. Maneja cinco tipos de consultas específicas hacia la plataforma Soccerway, estos son los próximos partidos, últimos partidos, historial *head-to-head* (H2H), partidos por período y tablas de posiciones.

Debido a que Soccerway genera su contenido de manera dinámica, se utilizó Playwright para operar un navegador en modo *headless*. Este servicio espera la carga completa del DOM relevante, maneja banners de *cookies* automáticamente y extrae el HTML renderizado. Finalmente, la librería BeautifulSoup procesa este HTML para estructurar la información fechas, equipos, resultados y URLs en formato JSON. Su diseño asegura que solo se ejecute ante un *Cache Miss*, protegiendo al sistema de bloqueos.

3.4. Metrics Service

Para evaluar el rendimiento bajo estrés, se implementó el Metrics Service, encargado de observar y registrar el comportamiento del sistema. Este módulo consolida las métricas vitales, generando archivos JSON detallados que incluyen la cantidad total de consultas, tasas de aciertos (*hit rate*) y fallos (*miss rate*), conteo de *evictions* (desalojos de memoria por falta de espacio), y registros de latencia y tiempos totales de ejecución.

4. Decisiones de Diseño y Tecnologías

- **FastAPI:** El uso de FastAPI se debió a sus características clave, como su rendimiento asincrónico y rapidez, haciéndolo ideal para este tipo de arquitecturas orientadas a microservicios.
- **Redis:** La elección de Redis como base de datos en memoria atiende a la necesidad de accesos ultrarrápidos y a su excelente soporte nativo para gestionar los tiempos de vida (TTL) de las respuestas del sistema.
- **Docker:** Para garantizar que el ecosistema pueda operar sin problemas y de manera idéntica en cualquier entorno, se decidió adoptar una estrategia de *containerización* mediante Docker, aislando cada servicio de forma independiente.

5. Pruebas y Resultados Preliminares

Para validar la correcta integración entre los módulos desarrollados hasta la fecha, se ejecutó una prueba de entorno local levantando simultáneamente el Generador de Tráfico y el Sistema de Caché (conectado a su instancia de Redis).

Tal como se observa en la Figura 1 y 2, el sistema se comportó exactamente según lo diseñado. Al enviar una consulta por primera vez (por ejemplo, `Q1:Cobreloa`), la API registró un `[MISS]`, indicando que los datos no existían en memoria y simulando la necesidad de invocar al Scraper.

Pocos milisegundos después, debido al comportamiento de la distribución de Zipf que fuerza la repetición de consultas, el Generador volvió a solicitar `Q1:Cobreloa`. Al estar dentro de la ventana del Tiempo de Vida (TTL) de 30 segundos, el sistema interceptó la consulta con éxito arrojando un `[HIT]`, recuperando los datos instantáneamente sin carga adicional.

```

ricardo@ricardo-VirtualBox: ~/grupo-4/Sistemas_de_cache
[MISS] La llave 'Q1:Cobreloa' no existe. Se llamaría al Scraper...
INFO: 127.0.0.1:39354 - "POST /api/consultas HTTP/1.1" 200 OK
[MISS] La llave 'Q1:U_de_Chile' no existe. Se llamaría al Scraper...
INFO: 127.0.0.1:39364 - "POST /api/consultas HTTP/1.1" 200 OK
[MISS] La llave 'Q5' no existe. Se llamaría al Scraper...
INFO: 127.0.0.1:39378 - "POST /api/consultas HTTP/1.1" 200 OK
[MISS] La llave 'Q1:Palestino' no existe. Se llamaría al Scraper...
INFO: 127.0.0.1:39388 - "POST /api/consultas HTTP/1.1" 200 OK
[MISS] La llave 'Q2:Colo_Colo' no existe. Se llamaría al Scraper...
INFO: 127.0.0.1:39402 - "POST /api/consultas HTTP/1.1" 200 OK
[MISS] La llave 'Q3:Cobreloa:U_de_Chile' no existe. Se llamaría al Scraper...
INFO: 127.0.0.1:39404 - "POST /api/consultas HTTP/1.1" 200 OK
[MISS] La llave 'Q1:Colo_Colo' no existe. Se llamaría al Scraper...
INFO: 127.0.0.1:39408 - "POST /api/consultas HTTP/1.1" 200 OK
[HIT] La llave 'Q1:Cobreloa' ya estaba en la Caché.
INFO: 127.0.0.1:39416 - "POST /api/consultas HTTP/1.1" 200 OK
[HIT] La llave 'Q1:Palestino' ya estaba en la Caché.
INFO: 127.0.0.1:39432 - "POST /api/consultas HTTP/1.1" 200 OK
[MISS] La llave 'Q1:Everton' no existe. Se llamaría al Scraper...
INFO: 127.0.0.1:39438 - "POST /api/consultas HTTP/1.1" 200 OK
[HIT] La llave 'Q1:Union_Espanola' ya estaba en la Caché.
INFO: 127.0.0.1:39442 - "POST /api/consultas HTTP/1.1" 200 OK
[HIT] La llave 'Q1:Colo_Colo' ya estaba en la Caché.
INFO: 127.0.0.1:39444 - "POST /api/consultas HTTP/1.1" 200 OK
[MISS] La llave 'Q2:U_Catolica' no existe. Se llamaría al Scraper...
INFO: 127.0.0.1:39456 - "POST /api/consultas HTTP/1.1" 200 OK
[HIT] La llave 'Q5' ya estaba en la Caché.
INFO: 127.0.0.1:39462 - "POST /api/consultas HTTP/1.1" 200 OK
[HIT] La llave 'Q1:Union_Espanola' ya estaba en la Caché.
INFO: 127.0.0.1:39476 - "POST /api/consultas HTTP/1.1" 200 OK
[HIT] La llave 'Q1:Cobreloa' ya estaba en la Caché.
INFO: 127.0.0.1:39482 - "POST /api/consultas HTTP/1.1" 200 OK
[MISS] La llave 'Q1:Audax_Italiano' no existe. Se llamaría al Scraper...
INFO: 127.0.0.1:45852 - "POST /api/consultas HTTP/1.1" 200 OK
[MISS] La llave 'Q2:U_de_Chile' no existe. Se llamaría al Scraper...
INFO: 127.0.0.1:45854 - "POST /api/consultas HTTP/1.1" 200 OK

```

Figura 1: Registro de la terminal mostrando la interacción en tiempo real y la resolución de aciertos (HIT) y desaciertos (MISS) en la caché.

```

ricardo@ricardo-VirtualBox: ~/grupo-4/Sistemas_de_cache
[4/20] Enviando Q1: {'tipo': 'Q1', 'equipo': 'U de Chile'}
-> Éxito: Caché respondió con status 200
[5/20] Enviando Q5: {'tipo': 'Q5'}
-> Éxito: Caché respondió con status 200
[6/20] Enviando Q1: {'tipo': 'Q1', 'equipo': 'Palestino'}
-> Éxito: Caché respondió con status 200
[7/20] Enviando Q2: {'tipo': 'Q2', 'equipo': 'Colo Colo'}
-> Éxito: Caché respondió con status 200
[8/20] Enviando Q3: {'tipo': 'Q3', 'equipo1': 'Cobreloa', 'equipo2': 'U de Chile'}
-> Éxito: Caché respondió con status 200
[9/20] Enviando Q1: {'tipo': 'Q1', 'equipo': 'Colo Colo'}
-> Éxito: Caché respondió con status 200
[10/20] Enviando Q1: {'tipo': 'Q1', 'equipo': 'Cobreloa'}
-> Éxito: Caché respondió con status 200
[11/20] Enviando Q1: {'tipo': 'Q1', 'equipo': 'Palestino'}
-> Éxito: Caché respondió con status 200
[12/20] Enviando Q1: {'tipo': 'Q1', 'equipo': 'Everton'}
-> Éxito: Caché respondió con status 200
[13/20] Enviando Q1: {'tipo': 'Q1', 'equipo': 'Union Espanola'}
-> Éxito: Caché respondió con status 200
[14/20] Enviando Q1: {'tipo': 'Q1', 'equipo': 'Colo Colo'}
-> Éxito: Caché respondió con status 200
[15/20] Enviando Q2: {'tipo': 'Q2', 'equipo': 'U Catolica'}
-> Éxito: Caché respondió con status 200
[16/20] Enviando Q5: {'tipo': 'Q5'}
-> Éxito: Caché respondió con status 200
[17/20] Enviando Q1: {'tipo': 'Q1', 'equipo': 'Union Espanola'}
-> Éxito: Caché respondió con status 200
[18/20] Enviando Q1: {'tipo': 'Q1', 'equipo': 'Cobreloa'}
-> Éxito: Caché respondió con status 200
[19/20] Enviando Q1: {'tipo': 'Q1', 'equipo': 'Audax Italiano'}
-> Éxito: Caché respondió con status 200
[20/20] Enviando Q2: {'tipo': 'Q2', 'equipo': 'U de Chile'}
-> Éxito: Caché respondió con status 200

Tráfico finalizado con éxito.
(venv) ricardo@ricardo-VirtualBox:~/grupo-4/generador_trafico$

```

Figura 2: Registro de la terminal mostrando el trafico generado.

6. Pruebas y Resultados Finales

Con el objetivo de evaluar el comportamiento de la caché de forma controlada y reproducible, se ejecutó una matriz de experimentos de presión. Cada configuración procesó 1000 solicitudes consecutivas, sin espera entre ellas, y se inició con una instancia limpia de Redis y las métricas reiniciadas. Esto permite comparar los resultados sin que datos o contadores de ejecuciones anteriores alteren las mediciones.

Las pruebas consideraron dos distribuciones de acceso: Uniforme, donde todas las claves tienen la misma probabilidad de ser seleccionadas, y Zipf con parámetro $\alpha = 1,1$, donde un subconjunto de claves concentra una mayor cantidad de accesos. Para cada distribución se evaluaron tres límites de memoria para Redis 2 MB, 5 MB, 10 MB y dos políticas de reemplazo LRU y LFU. En total, se evaluaron 12 configuraciones comparables.

Para aislar el efecto de la caché y provocar desalojos de forma consistente, las pruebas de presión utilizaron un espacio de 1000 claves sintéticas, con valores de 64 KiB por entrada y un TTL de 3600 segundos. De esta forma, los *evictions* observados responden principalmente al límite de memoria y a la política de reemplazo, no a la expiración temporal de las claves. En cada ejecución se registraron solicitudes totales, *hits*, *misses*, *hit rate*, latencia de extremo a extremo, *throughput* y cantidad de *evictions*.

Los experimentos de presión se complementan con pruebas funcionales realizadas sobre consultas reales Q1–Q5 hacia Soccerway. Mientras las pruebas funcionales validan la integración completa entre generador, caché y scraper, la matriz de presión permite comparar de manera justa el efecto de la distribución de tráfico, el tamaño de caché y la política de reemplazo.

6.1. Impacto de la Distribución de Tráfico y Tamaño de Caché (Uniforme)

Para comprender el impacto de la distribución de las consultas en el rendimiento del sistema, se ejecutó una serie de experimentos bajo una distribución Uniforme. En esta configuración, cada consulta del catálogo tiene exactamente la misma probabilidad de ser elegida por el Generador de Tráfico, lo que impide que la caché pueda predecir y retener eficientemente las claves más populares.

En el Cuadro 1 se resumen las métricas obtenidas tras inyectar 1000 solicitudes bajo distintos tamaños de memoria y políticas de remoción utilizando una distribución uniforme.

Tamaño	Política	Hits	Misses	Hit Rate	Evictions	p50	p95	Throughput
2 MB	LFU	7	993	0.7 %	983	8.76 ms	10.90 ms	109.08 rps
2 MB	LRU	8	992	0.8 %	981	8.74 ms	10.98 ms	109.37 rps
5 MB	LFU	40	960	4.0 %	911	8.62 ms	10.33 ms	111.82 rps
5 MB	LRU	39	961	3.9 %	912	8.84 ms	10.87 ms	108.47 rps
10 MB	LFU	82	918	8.2 %	805	8.61 ms	10.87 ms	111.19 rps
10 MB	LRU	87	913	8.7 %	800	9.12 ms	11.74 ms	104.50 rps

Cuadro 1: Hits, misses, latencia p50/p95, throughput y evictions tras 1000 solicitudes bajo distribución Uniforme.

Análisis de Tamaño y Desalojos (Evictions): Como se observa en los resultados obtenidos, el tamaño estricto de la memoria determina el comportamiento de los datos. Al limitar Redis a solo 2 MB, el sistema enfrenta alta presión, forzando 983 desalojos con LFU y 981 con LRU. Al incrementar la capacidad a 10 MB, los desalojos disminuyen a 805 y 800, permitiendo que una mayor cantidad de datos permanezca en memoria.

Patrones de Acceso en Distribucion Uniforme: La distribución Uniforme demostró ser el peor escenario para la caché. Al no existir consultas predominantemente populares, el *Hit Rate* máximo alcanzó 8.7%. Prácticamente cada solicitud resultó en un *Cache Miss*; en una ejecución funcional esto obligaría a invocar constantemente al Scraper Service.

6.2. Impacto de la Distribución de Tráfico y Tamaño de Caché (Zipf)

Para complementar el análisis anterior y simular con mayor fidelidad el comportamiento de una plataforma de alto tráfico en el mundo real, se ejecutó una segunda serie de experimentos bajo una distribución de Zipf. En esta configuración (parámetro $\alpha = 1,1$), un conjunto reducido de consultas concentra la gran mayoría de las solicitudes de los usuarios, lo que permite observar cómo la caché predice y retiene eficientemente las claves más populares.

En el Cuadro 2 se resumen las métricas obtenidas tras inyectar 1000 solicitudes bajo distintos tamaños de memoria y políticas de remoción utilizando una distribución zipf.

Tamaño	Política	Hits	Misses	Hit Rate	Evictions	p50	p95	Throughput
2 MB	LFU	397	603	39.7 %	593	13.47 ms	18.24 ms	71.10 rps
2 MB	LRU	262	738	26.2 %	728	14.62 ms	20.09 ms	65.49 rps
5 MB	LFU	584	416	58.4 %	367	8.70 ms	10.94 ms	109.64 rps
5 MB	LRU	509	491	50.9 %	443	8.52 ms	10.09 ms	112.80 rps
10 MB	LFU	665	335	66.5 %	222	8.47 ms	10.22 ms	113.10 rps
10 MB	LRU	643	357	64.3 %	244	8.48 ms	10.33 ms	112.95 rps

Cuadro 2: Hits, misses, latencia p50/p95, throughput y evictions tras 1000 solicitudes bajo distribución Zipf.

Análisis de Tamaño y Desalojos (Evictions): A diferencia del escenario uniforme, Zipf y el tamaño de la caché muestran una correlación directa sobre el rendimiento. En 2 MB se registraron 593 desalojos bajo LFU y 728 bajo LRU. A 10 MB, los desalojos disminuyeron a 222 y 244. Esta holgura redujo la rotación de datos y permitió mantener claves repetitivas.

Patrones de Acceso en Distribución de Zipf: Bajo Zipf, LFU supera a LRU en *Hit Rate* en todos los tamaños. En 2 MB logra 39.7 % frente a 26.2 %; en 5 MB, 58.4 % frente a 50.9 %; y en 10 MB, 66.5 % frente a 64.3 %. Esto evidencia que LFU protege mejor las claves solicitadas con mayor frecuencia.

6.3. Comparación: Distribución Uniforme vs. Distribución de Zipf

Al contrastar los resultados de los experimentos de la distribución Uniforme con los de la distribución de Zipf se evidencia el impacto crítico del patrón de acceso en el desempeño de una arquitectura de caché.

Bajo una distribución Uniforme, la incapacidad de la caché para priorizar información se hace evidente. Dado que todas las claves tienen la misma probabilidad matemática de ser solicitadas, tanto LRU como LFU operan a ciegas, expulsando datos que tienen la misma posibilidad de volver a ser consultados que los recién ingresados. Esto provoca un escenario de reemplazo constante de memoria (*thrashing*), resultando en tasas de acierto insignificantes y obligando a depender casi permanentemente del servicio subyacente de *Scraping*.

En contraparte, la distribución de Zipf favorece la mecánica de la memoria caché. Al concentrar el tráfico en una cantidad reducida de elementos, emulando la alta demanda sobre equipos deportivos populares, el sistema logra aprender qué claves retener. Esto se refleja en cómo el *Hit Rate* escala sostenidamente hasta un 12.7 % al contar con una memoria mas grande para alojar el grupo de claves más frecuentes. En este escenario, la caché cumple su propósito principal el cual es absorber las ráfagas repetitivas de solicitudes y reducir la latencia general.

Como corrección de los resultados validados, el máximo *Hit Rate* observado en Zipf fue 66.5 % con LFU y 10 MB. Las Figuras 3 y 4 resumen la diferencia entre ambos patrones de acceso.

Tasa de aciertos por distribución, tamaño y política

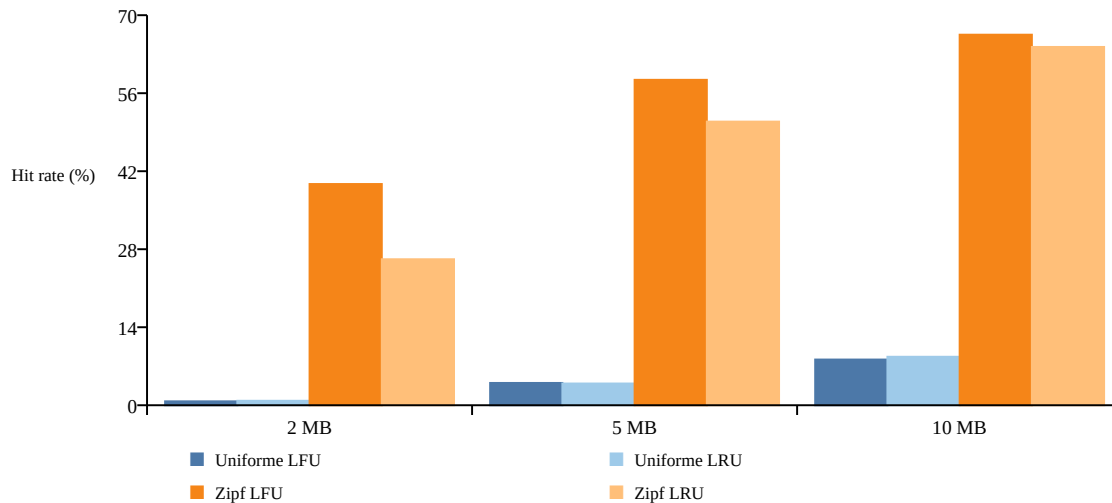


Figura 3: Hit rate según distribución, tamaño y política de reemplazo.

Desalojos de caché en 1000 solicitudes

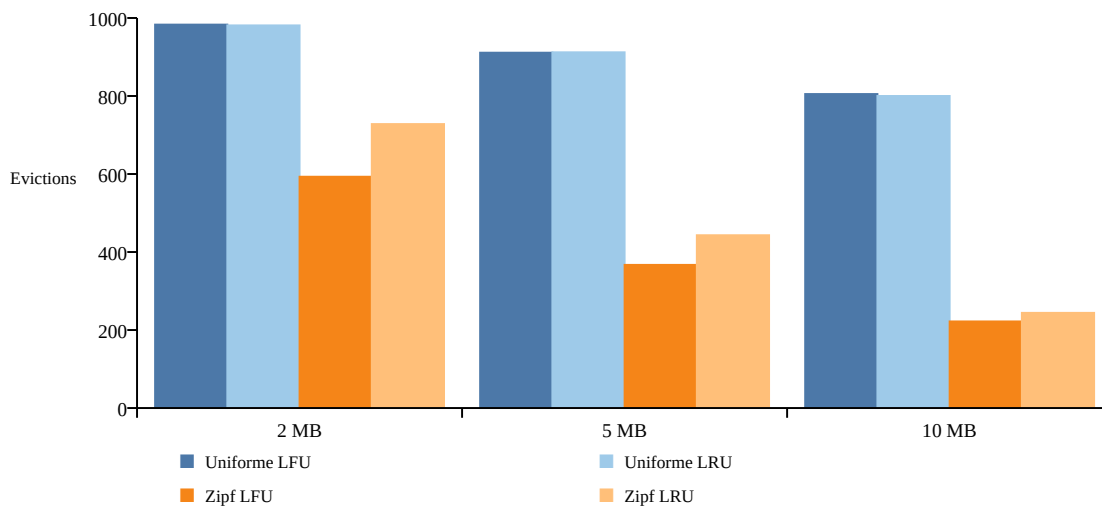


Figura 4: Evictions en 1000 solicitudes por configuración.

6.4. Evaluación del Impacto del Time To Live

El parámetro del tiempo de vida dictamina el tiempo exacto que un registro permanecerá válido dentro de Redis antes de expirar. Dado que las consultas tienen características muy distintas, elegir correctamente cuándo expiran los datos es vital para que el sistema responda rápido sin entregar información obsoleta.

- **Consultas de Baja Variabilidad (Q3 - Historial de Enfrentamientos):** El historial de partidos históricos entre dos equipos no sufre modificaciones hasta el instante en que finaliza un nuevo encuentro entre ellos. Para este tipo de datos casi estáticos, resulta altamente conveniente configurar un TTL prolongado, por ejemplo 24 horas o más. Un TTL mayor en este caso maximiza el *Hit Rate* y evita sobrecargar al *Scraper*, sin riesgo de entregar información caducada.

- **Consultas Altamente Dinámicas (Q2 - Últimos Partidos y Q5 - Tabla de Posiciones):** Estas consultas requieren reflejar el estado actual y preciso del campeonato. Durante la jornada deportiva, los resultados y posiciones cambian de forma constante. En este escenario, un TTL prolongado perjudicaría al sistema entregando información obsoleta y afectando la experiencia del usuario. Se recomienda un TTL mucho más agresivo, por ejemplo de 5 a 10 minutos, que fuerce la expiración y obligue al sistema a consultar datos frescos desde la fuente externa.
- **Consultas de Variabilidad Media (Q1 - Próximos Partidos y Q4 - Partidos por Fecha):** La programación futura de los encuentros suele ser estable en el transcurso de una semana, aunque es susceptible a cambios de cancha o suspensiones. Estas consultas se benefician de un TTL intermedio, por ejemplo de algunas horas, proporcionando un punto de equilibrio entre una respuesta de baja latencia desde la caché y la detección de eventuales reprogramaciones.

En síntesis, un TTL global mal configurado impactará directamente en las métricas. Un TTL excesivamente corto provocará la expiración temprana de registros, aumentando artificialmente el *Miss Rate* y degradando el rendimiento general. Por lo tanto, el sistema debe contemplar tiempos de vida ajustables según las características intrínsecas de cada tipo de consulta.

La recomendación anterior fue validada experimentalmente para Q1–Q5. Con TTL de 3 segundos la segunda solicitud ocurrió después de la expiración y produjo un nuevo miss; con TTL de 60 segundos la segunda solicitud fue un hit. En ambos casos se mantuvieron dos solicitudes por configuración.

Consulta	TTL (s)	Hits	Misses	Expired keys
Q1	3 / 60	0 / 1	2 / 1	1 / 0
Q2	3 / 60	0 / 1	2 / 1	1 / 0
Q3	3 / 60	0 / 1	2 / 1	1 / 0
Q4	3 / 60	0 / 1	2 / 1	1 / 0
Q5	3 / 60	0 / 1	2 / 1	1 / 0

Cuadro 3: Validación experimental del efecto de TTL por tipo de consulta.

6.5. Validación Funcional y Resiliencia

Las pruebas funcionales confirmaron respuestas válidas para Q1, Q2, Q3, Q4 y Q5 con consultas reales hacia Soccerway. También se ejecutaron fallas controladas: un payload inválido retorna HTTP 400 antes de consultar Redis; un scraper no disponible o timeout retorna HTTP 502 y no se almacena la respuesta; y Redis no disponible retorna HTTP 503. La caída del servicio de métricas no interrumpe la respuesta principal, ya que la telemetría se registra como operación de mejor esfuerzo.

Como referencia de la ejecución real, las pruebas funcionales de 40 solicitudes con 5 MB registraron en Uniforme/LRU un hit rate de 35.0 %, p50 de 3194.32 ms y p95 de 4881.44 ms. En Zipf/LRU el hit rate aumentó a 62.5 % y el p50 bajó a 9.46 ms, mientras que el p95 se mantuvo alto por el costo de los misses que requieren scraping. Esto relaciona directamente la concentración de accesos con la latencia observada.

6.6. Evidencia de las consultas Q1–Q5

Para complementar los resultados experimentales, se realizaron consultas individuales mediante solicitudes HTTP POST al servicio de caché. Las siguientes capturas muestran fragmentos de las respuestas JSON correspondientes a los cinco tipos de consulta implementados.

```
PS C:\Users\conno\Documents\cache\AllYouCanCacheSoccerway> Invoke-RestMethod -Method Post -Uri "http://localhost:5001/api/consultas" -ContentType "application/json" -Body '{"tipo":"Q2","slug":"colo-colo","equipo_id":"th4HPiws"}' | ConvertTo-Json -Depth 10
{
  "status": "miss",
  "origen": "scraper",
  "datos": {
    "equipo": "colo-colo",
    "ultimos_partidos": [
      {
        "fecha": "06.09.20:00",
        "hora": "20:00",
        "competicion": "Liga de Primera",
        "equipo_local": "Huachipato",
        "equipo_visitante": "Colo Colo",
        "goles_local": "0",
        "goles_visitante": "0",
        "url_partido": "https://cl.soccerway.com/detalle-del-partido/colo-colo-th4HPiws/huachipato-SlhZpIaP/?mid=CY6"
      },
      {
        "fecha": "30.08.21:30",
        "hora": "21:30",
        "competicion": "Liga de Primera",
        "equipo_local": "Colo Colo",
        "equipo_visitante": "Audax",
        "goles_local": "5",
        "goles_visitante": "1",
        "url_partido": "https://cl.soccerway.com/detalle-del-partido/a-italiano-rN3Jxc8g/colo-colo-th4HPiws/?mid=rep"
      },
      {
        "fecha": "23.08.19:00",
        "hora": "19:00",
        "competicion": "Liga de Primera",
        "equipo_local": "U. De Chile",
        "equipo_visitante": "Colo Colo",
        "goles_local": "1",
        "goles_visitante": "2",
        "url_partido": "https://cl.soccerway.com/detalle-del-partido/colo-colo-th4HPiws/u-de-chile-xW771C6U/?mid=8d2"
      }
    ]
  }
}
```

Figura 5: Consulta Q1 sobre los proximos partidos de Colo Colo

La consulta Q1 permite recuperar los próximos encuentros programados de Colo Colo. La respuesta incluye fecha, hora y equipos local y visitante, permitiendo identificar la programación futura disponible en la fuente.

```
PS C:\Users\conno\Documents\cache\AllYouCanCacheSoccerway> Invoke-RestMethod -Method Post -Uri "http://localhost:5001/api/consultas" -ContentType "application/json" -Body '{"tipo":"Q2","slug":"colo-colo","equipo_id":"th4HPiws"}' | ConvertTo-Json -Depth 10
{
  "status": "miss",
  "origen": "scraper",
  "datos": {
    "equipo": "colo-colo",
    "ultimos_partidos": [
      {
        "fecha": "06.09.20:00",
        "hora": "20:00",
        "competicion": "Liga de Primera",
        "equipo_local": "Huachipato",
        "equipo_visitante": "Colo Colo",
        "goles_local": "0",
        "goles_visitante": "0",
        "url_partido": "https://cl.soccerway.com/detalle-del-partido/colo-colo-th4HPiws/huachipato-SlhZpIaP/?mid=CY6"
      },
      {
        "fecha": "30.08.21:30",
        "hora": "21:30",
        "competicion": "Liga de Primera",
        "equipo_local": "Colo Colo",
        "equipo_visitante": "Audax",
        "goles_local": "5",
        "goles_visitante": "1",
        "url_partido": "https://cl.soccerway.com/detalle-del-partido/a-italiano-rN3Jxc8g/colo-colo-th4HPiws/?mid=rep"
      },
      {
        "fecha": "23.08.19:00",
        "hora": "19:00",
        "competicion": "Liga de Primera",
        "equipo_local": "U. De Chile",
        "equipo_visitante": "Colo Colo",
        "goles_local": "1",
        "goles_visitante": "2",
        "url_partido": "https://cl.soccerway.com/detalle-del-partido/colo-colo-th4HPiws/u-de-chile-xW771C6U/?mid=8d2"
      }
    ]
  }
}
```

Figura 6: Consulta Q2 sobre los ultimos resultados de Colo Colo

La consulta Q2 devuelve partidos recientes de Colo Colo. En el fragmento mostrado se identifican las fechas, los equipos participantes y sus respectivos marcadores, junto con la competición y el enlace al encuentro en Soccerway.

```
PS C:\Users\conno\Documents\cache\AllYouCanCacheSoccerway> Invoke-RestMethod -Method Post -Uri "http://localhost:5001/api/consultas" -ContentType "application/json" -Body '{"tipo":"Q3","slug1":"colo-colo","id1":"th4HPiws","slug2":"u-de-chile","id2":"xw771C6U"}' | ConvertTo-Json -Depth 10
{
  "status": "miss",
  "origen": "scraper",
  "datos": {
    "equipo_1": "Colo Colo",
    "equipo_2": "U. De Chile",
    "enfrentamientos": [
      {
        "fecha": "23.08.26",
        "competicion": "Liga de Primera (Chile)",
        "equipo_local": "U. De Chile",
        "equipo_visitante": "Colo Colo",
        "goles_local": "1",
        "goles_visitante": "2",
        "url_partido": "https://cl.soccerway.com/detalle-del-partido/colo-colo-th4HPiws/u-de-chile-xw771C6U/?mid=8d261tCL"
      },
      {
        "fecha": "01.03.26",
        "competicion": "Liga de Primera (Chile)",
        "equipo_local": "Colo Colo",
        "equipo_visitante": "U. De Chile",
        "goles_local": "0",
        "goles_visitante": "1",
        "url_partido": "https://cl.soccerway.com/detalle-del-partido/colo-colo-th4HPiws/u-de-chile-xw771C6U/?mid=xwMJ03tJ"
      },
      {
        "fecha": "14.09.25",
        "competicion": "Supercopa (Chile)",
        "equipo_local": "Colo Colo",
        "equipo_visitante": "U. De Chile",
        "goles_local": "0",
        "goles_visitante": "3",
        "url_partido": "https://cl.soccerway.com/detalle-del-partido/colo-colo-th4HPiws/u-de-chile-xw771C6U/?mid=fJgvXY5U"
      }
    ]
  }
}
```

Figura 7: Consulta Q3 sobre el historial entre Colo Colo y Universidad de Chile

La consulta Q3 recupera enfrentamientos directos entre Colo Colo y Universidad de Chile. La respuesta mostrada incluye encuentros de distintas fechas, indicando competición, local, visitante y resultado. Esto evidencia la obtención de información histórica para el par de equipos consultado.

```
PS C:\Users\conno\Documents\cache\AllYouCanCacheSoccerway> Invoke-RestMethod -Method Post -Uri "http://localhost:5001/api/consultas" -ContentType "application/json" -Body '{"tipo":"Q4","fecha_inicio":"2026-09-07","fecha_fin":"2026-09-14"}' | ConvertTo-Json -Depth 10
{
  "status": "miss",
  "origen": "scraper",
  "datos": {
    "periodo": "2026-09-07 a 2026-09-14",
    "partidos": [
      {
        "jornada": null,
        "fecha": "14.09. 23:30",
        "hora": "23:30",
        "equipo_local": "U. La Calera",
        "equipo_visitante": "Limache",
        "goles_local": null,
        "goles_visitante": null,
        "url_partido": "https://cl.soccerway.com/detalle-del-partido/Limache-YPTUuyqJ/u-la-calera-M34g8uLJ/?mid=KxkYBrme"
      },
      {
        "jornada": null,
        "fecha": "13.09. 20:30",
        "hora": "20:30",
        "equipo_local": "Colo Colo",
        "equipo_visitante": "Dep. Concepci n",
        "goles_local": "1",
        "goles_visitante": "1",
        "url_partido": "https://cl.soccerway.com/detalle-del-partido/colo-colo-th4HPiws/dep-concepcion-GKJrZE7I/?mid=dduqYY3c"
      },
      {
        "jornada": null,
        "fecha": "13.09. 18:00",
        "hora": "18:00",
        "equipo_local": "La Serena",
        "equipo_visitante": "U. De Chile",
        "goles_local": "0",
        "goles_visitante": "3",
        "url_partido": "https://cl.soccerway.com/detalle-del-partido/la-serena-d6kYgx8t/u-de-chile-xw771C6U/?mid=0AePD4Ir"
      },
      {
        "jornada": null,

```

Figura 8: Consulta Q4 sobre los partidos entre el 7 y 14 de septiembre de 2026 de Colo Colo

La consulta Q4 solicita los partidos comprendidos entre el 7 y el 14 de septiembre de 2026. El fragmento de la respuesta incluye encuentros programados y encuentros con marcadores disponibles. Los valores nulos en los goles indican que no se obtuvo un resultado para ese encuentro, y no deben interpretarse como un marcador de cero.

```
PS C:\Users\conno\Documents\cache\AllYouCanCacheSoccerway> Invoke-RestMethod -Method Post -Uri "http://localhost:5001/api/consultas" -ContentType "application/json" -Body '{"tipo":"Q5"}' | ConvertTo-Json -Depth 10
{
  "status": "miss",
  "origen": "scraper",
  "datos": {
    "liga": "Primera Division Chile",
    "tabla": [
      {
        "posicion": 1,
        "equipo": "Colo Colo",
        "equipo_id": "th4HPiws",
        "equipo_slug": "colo-colo",
        "partidos_jugados": 23,
        "ganados": 17,
        "empatados": 3,
        "perdidos": 3,
        "goles_favor": 48,
        "goles_contra": 22,
        "diferencia_goles": 26,
        "puntos": 54
      },
      {
        "posicion": 2,
        "equipo": "U. Católica",
        "equipo_id": "Qe432hi0",
        "equipo_slug": "u-catolica",
        "partidos_jugados": 23,
        "ganados": 13,
        "empatados": 3,
        "perdidos": 7,
        "goles_favor": 50,
        "goles_contra": 33,
        "diferencia_goles": 17,
        "puntos": 42
      },
      {
        "posicion": 3,
        "equipo": "U. De Chile",
        "equipo_id": "xW771C6U",
        "equipo_slug": "u-de-chile",

```

Figura 9: Consulta Q5 sobre la tabla completa de posiciones

La consulta Q5 recupera la tabla de posiciones de la Liga de Primera de Chile. La captura muestra un fragmento de las entradas devueltas, con posición, equipo, partidos jugados, victorias, empates, derrotas, goles a favor, goles en contra, diferencia de goles y puntos.

6.7. Registro de Archivos de Experimentación

Los datos fueron extraídos directamente de los siguientes archivos en formato JSON. Estos documentos fueron generados y almacenados automáticamente por el *Metrics Service* tras la finalización de cada prueba de estrés:

- Caché de 2 MB (LFU): pressure-uniform-2mb-lfu-20260912-193010.json
- Caché de 2 MB (LRU): pressure-uniform-2mb-lru-20260912-192829.json
- Caché de 5 MB (LFU): pressure-uniform-5mb-lfu-20260912-193043.json
- Caché de 5 MB (LRU): pressure-uniform-5mb-lru-20260912-192902.json
- Caché de 10 MB (LFU): pressure-uniform-10mb-lfu-20260912-193116.json
- Caché de 10 MB (LRU): pressure-uniform-10mb-lru-20260912-192936.json
- Caché de 2 MB (LFU): pressure-zipf-2mb-lfu-20260912-183919.json
- Caché de 2 MB (LRU): pressure-zipf-2mb-lru-20260912-182126.json
- Caché de 5 MB (LFU): pressure-zipf-5mb-lfu-20260912-184009.json
- Caché de 5 MB (LRU): pressure-zipf-5mb-lru-20260912-182241.2.json
- Caché de 10 MB (LFU): pressure-zipf-10mb-lfu-20260912-184055.2.json
- Caché de 10 MB (LRU): pressure-zipf-10mb-lru-20260912-182326.2.json

Cada archivo contiene el registro íntegro de las 1000 solicitudes inyectadas por el Generador de Tráfico, especificando la configuración exacta del contenedor, los *Cache Hits*, *Misses*, cantidad de *evictions* (desalojos) y la latencia experimentada en cada consulta individual.

6.8. Métricas Complementarias de Rendimiento

Además de la cantidad total de desalojos presentada en los cuadros anteriores, se calculó la tasa de remoción como *evictions* por minuto. Esta métrica permite comparar la intensidad de reemplazo aun cuando las corridas tengan duraciones levemente distintas. Los valores se obtuvieron a partir de la diferencia de contadores Redis entre el inicio y término de cada experimento.

Distribución	Tamaño	LFU evictions/min	LRU evictions/min
Uniforme	2 MB	6433.31	6437.34
Uniforme	5 MB	6111.93	5935.45
Uniforme	10 MB	5370.68	5015.93
Zipf	2 MB	2529.90	2860.80
Zipf	5 MB	2414.32	2998.36
Zipf	10 MB	1506.53	1653.54

Cuadro 4: Tasa de evictions por minuto en las pruebas de presión.

La tasa de remoción confirma la tendencia observada previamente: Uniforme fuerza una rotación alta de entradas incluso con 10 MB. Zipf reduce de forma importante los desalojos porque reutiliza las claves populares; LFU presenta la menor tasa en los tres tamaños bajo esta distribución.

También se evaluaron tiempo promedio de scraping y *cache efficiency* en pruebas funcionales reales. La eficiencia se calcula como $(hits \cdot t_{cache} - misses \cdot t_{fuente}) / total$. Su valor negativo expresa que los misses externos dominan el costo promedio; el valor menos negativo de Zipf muestra que una mayor proporción de respuestas rápidas desde caché reduce dicho costo.

Distribución	Política	Hit Rate	Scraping prom.	Cache efficiency	Errores
Uniforme	LRU	35.0 %	3890.03 ms	-2528.29 ms	0
Zipf	LRU	62.5 %	3771.80 ms	-1414.11 ms	0

Cuadro 5: Métricas funcionales con Soccerway, 40 solicitudes y caché de 5 MB.

6.9. Trazabilidad de Resultados Finales

Para evitar ambigüedad entre corridas exploratorias y resultados finales, las tablas y gráficos de este informe se construyeron desde `experiments/analysis/summary.csv`. La cobertura de la matriz está verificada en `experiments/analysis/validation.json`; las respuestas funcionales Q1–Q5 se documentan en `functional_evidence.json`; y la evidencia de fallas controladas se registra en `resilience.json`. Los archivos enumerados en la subsección anterior se conservan como registro histórico de ejecuciones, mientras que estas fuentes consolidadas corresponden a los valores reportados en el análisis final.

7. Conclusiones

El sistema implementa el flujo completo Generador → Caché → Scraper → Caché → respuesta, junto con el registro de métricas. Los experimentos confirman que Zipf favorece la caché al reutilizar claves populares, mientras que Uniforme produce muchos misses y evictions.

El aumento de 2 MB a 10 MB redujo desalojos y elevó el hit rate. LFU mostró su ventaja más clara bajo Zipf y poca memoria, pues protege entradas consultadas frecuentemente; sin embargo, LRU y LFU tuvieron throughput similar, por lo que la política debe evaluarse junto con hit rate y evictions.

La evaluación TTL confirma el equilibrio entre rendimiento y frescura: consultas dinámicas requieren expiración menor, mientras que datos históricos pueden permanecer más tiempo en caché. El código, Docker Compose, instrucciones de despliegue y evidencia experimental están disponibles en <https://giteit.udp.cl/CIT2011/2026-2/seccion-2/grupo-4>.

Declaración sobre uso de herramientas de Inteligencia Artificial

Durante el desarrollo de esta tarea se utilizaron herramientas de Inteligencia Artificial como apoyo puntual para depuración de errores, mejora de sintaxis y claridad del código, revisión de documentación y orientación técnica. Las decisiones de diseño, implementación, ejecución de experimentos, validación de resultados y elaboración final del trabajo fueron realizadas y verificadas por los integrantes del equipo.